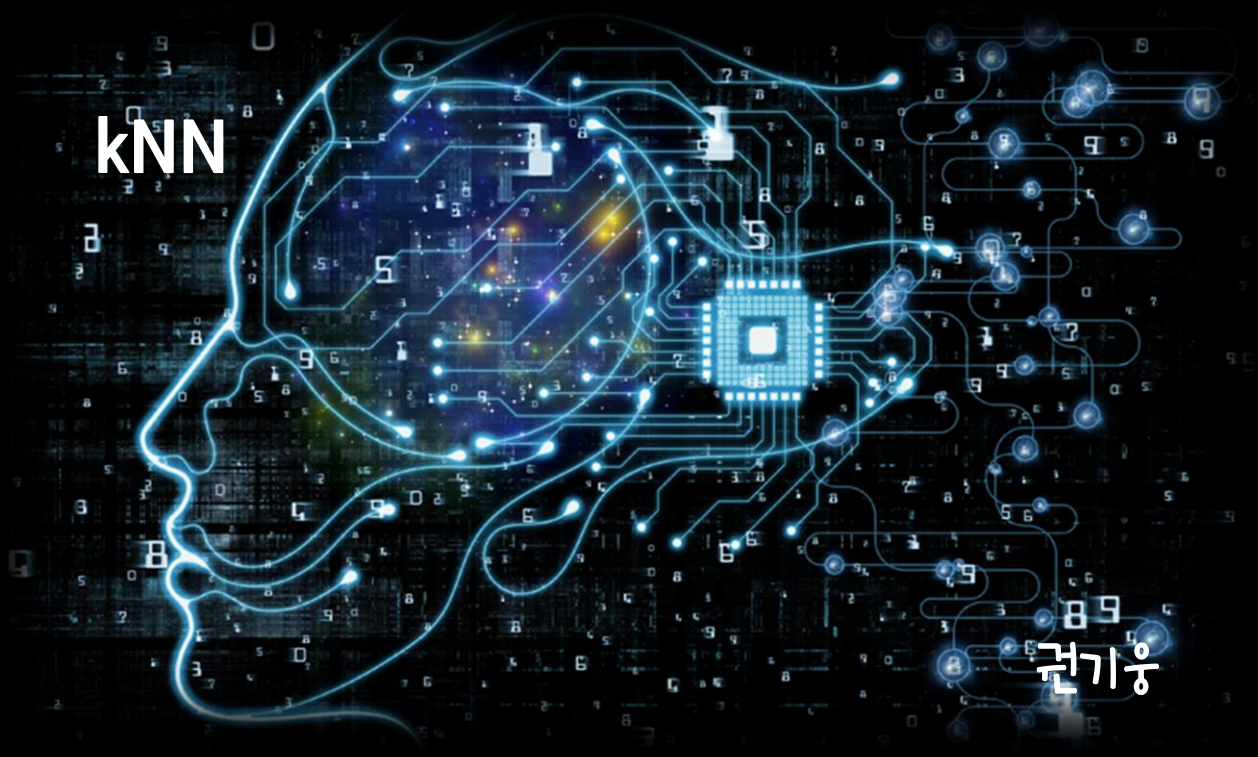
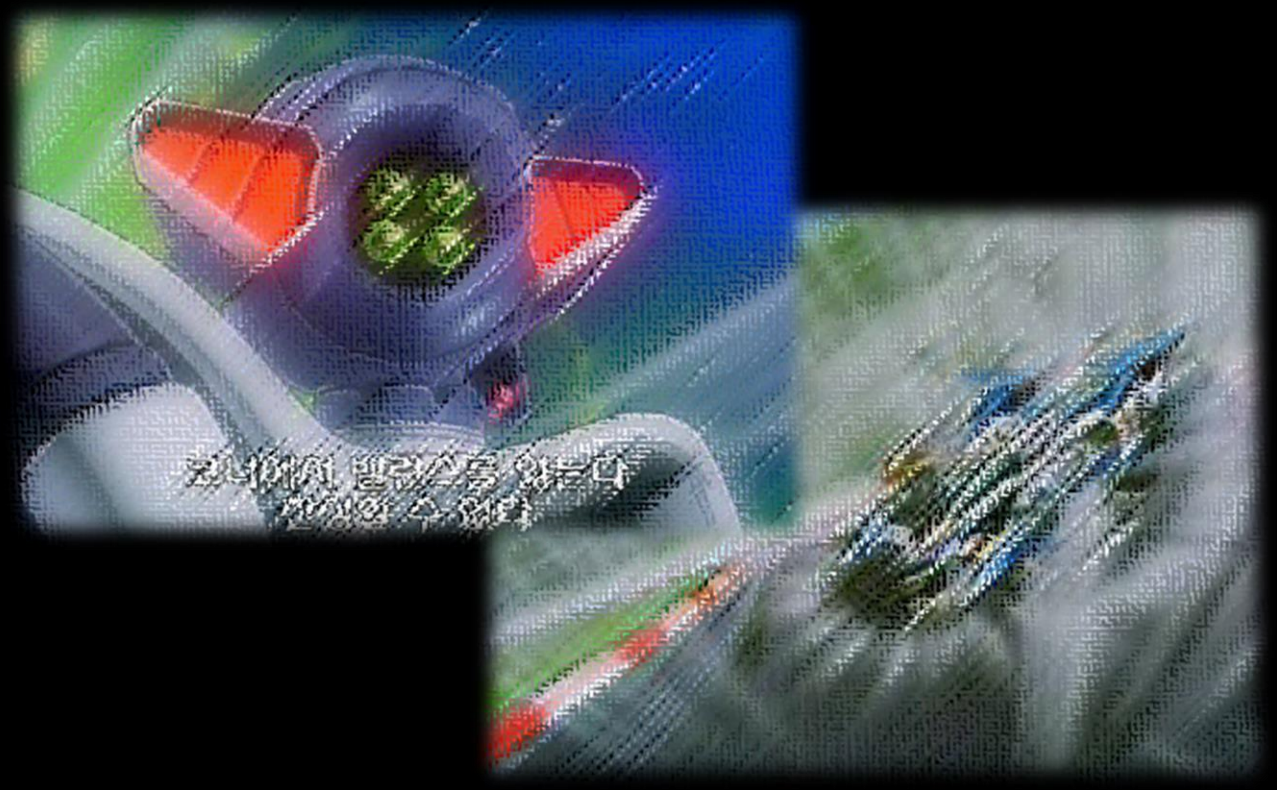

kNN
권기웅



1. kNN 알고리즘

1.1 kNN 알고리즘

1.1.1 k-Nearest Neighbors Algorithm

영화			
영화	격투씬	욕씬	장르
A	80	20	액션
B	95	5	액션
C	10	90	조폭
D	90	10	조폭
E	5	95	액션
Z	30	50	?

- 영화 A 는 액션, 영화 B 는 액션, ...이라고 컴퓨터에게 학습시키고
- 영화 Z 의 장르를 구하는 문제
- A ~ E 중에 Z 와 유사한 데이터 상위 k 개를 추출
- 다수결로 최종 결정

Z와 가장 가까운 점 k개 추출
k = 3이라면 C, E, A
C : 조폭
E : 액션
A : 액션
조폭1, 액션2
다수결로 처리 => Z는 액션

2. 영화 장르 구하기

2.1 kNN.java

2.1.1 거리 계산하는 메소드

```
// (테스트x - 샘플x)^2 + (테스트y - 샘플y)^2 + (테스트z - 샘플z)^2의 양의 제곱근
// 거리계산해서 double[]로
// p : double[] testData, ArrayList<double[]> datas
private static double[] calculateDistance(double[] testData, ArrayList<double[]>
    double[] distance = new double[datas.size()];
    double[] data;
    double sum;
    for (int i = 0; i < datas.size(); i++) {
        sum = 0;
        data = datas.get(i);
        for (int j = 0; j < data.length; j++) {
            sum += ((testData[j] - data[j]) * (testData[j] - data[j]));
        }
        distance[i] = Math.sqrt(sum);
    }
    return distance;
}
```

2.1.2 영화 장르들 추출하는 메소드

```
// 라벨(장르)을 중복제거된 String[]로
private static String[] getLabelSet(ArrayList<String> labels) {
    HashSet<String> labelSet = new HashSet<>(labels);
    String[] labelSetAr = labelSet.toArray(new String[labelSet.size()]);
    return labelSetAr;
}
```

2.1.3 kNN

```java
// kNN적용해서 결과를 String으로
// p : double[] testData, HashMap<String, ArrayList> trainData, int k
@SuppressWarnings({ "unchecked", "rawtypes" })
public static String do_kNN(double[] testData, HashMap<String, ArrayList> trainDa
    // 학습된 씬 횟수 들
    ArrayList<double[]> trainDatas = trainData.get("datas");

    // 학습된 장르 들
    ArrayList<String> trainLabels = trainData.get("labels");

    // 학습된 것들과 테스트할 데이터간의 거리계산
    double[] distance = calculateDistance(testData, trainDatas);

    // 거리계산된 배열의 데이터를 오름차순 정렬해서 새로운 배열
    double[] sortedDistance = Arrays.copyOf(distance, distance.length);
    Arrays.sort(sortedDistance);
```

```java
    // k개 만큼 세서, HashMap<String, Integer>으로 결과 만들기
        // "액션" : x번
        // "조폭" : x번
    HashMap<String, Integer> resultMap = new HashMap<>();
    String[] labelSet = getLabelSet(trainLabels);
    for (String s : labelSet) {
        resultMap.put(s, 0);
    }
    for (int i = 0; i < k; i++) {
        for (int j = 0; j < distance.length; j++) {
            if (sortedDistance[i] == distance[j]) {
                resultMap.replace(
                    trainLabels.get(j), resultMap.get(trainLabels.get(j)) + 1);
                break;
            }
        }
    }
```

```java
    // 횟수 제일 많은거 라벨(장르)을 리턴
    int max = 0;
    String result = null;
    for (String s : labelSet) {
        if (max <= resultMap.get(s)) {
            max = resultMap.get(s);
            result = s;
        }
    }
    System.out.println(resultMap);
    return result;
}
```

2.2 kNNMain.java

2.2.1 영화 데이터 만들어 학습시키는 메소드

```
@SuppressWarnings("rawtypes")
private static HashMap<String, ArrayList> trainMovieData() {
    HashMap<String, ArrayList> trainData = new HashMap<>();
    ArrayList<double[]> datas = new ArrayList<>();
    datas.add(new double[] { 80, 20 });
    datas.add(new double[] { 95, 5 });
    datas.add(new double[] { 10, 90 });
    datas.add(new double[] { 90, 10 });
    datas.add(new double[] { 5, 95 });

    ArrayList<String> labels = new ArrayList<>();
    labels.add("액션");
    labels.add("액션");
    labels.add("조폭");
    labels.add("조폭");
    labels.add("액션");

    trainData.put("labels", labels);
    trainData.put("datas", datas);
    return trainData;
}
```

2.2.2 main()

```
@SuppressWarnings("rawtypes")
public static void main(String[] args) {
    HashMap<String, ArrayList> trainData = trainMovieData();

    Scanner s = new Scanner(System.in);
    double[] testData = new double[2];

    System.out.print("싸움 : ");
    testData[0] = s.nextDouble();

    System.out.print("욕 : ");
    testData[1] = s.nextDouble();

    System.out.print("k : ");
    int k = s.nextInt();
    String result = kNN.do_kNN(testData, trainData, k);
    System.out.println(result);

    s.close();
}
```

2.3 실행

```
싸움 : 50
욕 : 30
k : 3
{조폭=1, 액션=2}
액션
```

3. 결혼 정보회사 인기도 구하기

3.1 kNN.java

3.1.1 파일에서 읽어오는 메소드

```
@SuppressWarnings("rawtypes")
public static HashMap<String, ArrayList> trainFromFile(String file) {
    HashMap<String, ArrayList> trainData = new HashMap<>();
    ArrayList<double[]> datas = new ArrayList<>();
    ArrayList<String> labels = new ArrayList<>();
    FileReader fr = null;
    try {
        fr = new FileReader(file);
        BufferedReader br = new BufferedReader(fr);
        String line;            String[] ar;            double[] dAr;
        while ((line = br.readLine()) != null) {
            ar = line.split("\t");
            dAr = new double[ar.length - 1];
            for (int i = 0; i < dAr.length; i++) {
                dAr[i] = Double.parseDouble(ar[i]);
            }
            datas.add(dAr);
            labels.add(ar[ar.length-1]);
        }
        trainData.put("labels", labels);
        trainData.put("datas", datas);
        return trainData;
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    } finally {
        try {fr.close();} catch (Exception e) {}
    }
}
```

3.1.2 학습 데이터 정규화 시키는 메소드

```java
// 학습 데이터 정규화
// p : HashMap<String, ArrayList> trainData
@SuppressWarnings({ "rawtypes", "unchecked" })
public static HashMap<String, ArrayList> normalizeTrainData(HashMap<String, Array
    HashMap<String, ArrayList> normalizedTrainData = new HashMap<>();
    ArrayList<Double> minVals = new ArrayList<>();
    ArrayList<Double> ranges = new ArrayList<>();
    ArrayList<double[]> datas = trainData.get("datas");
    double min;
    double max;
    for (int i = 0; i < datas.get(0).length; i++) {
        min = datas.get(0)[i];
        max = datas.get(0)[i];
        for (int j = 0; j < datas.size(); j++) {
            if (min > datas.get(j)[i]) {
                min = datas.get(j)[i];
            }
            if (max < datas.get(j)[i]) {
                max = datas.get(j)[i];
            }
        }
        minVals.add(min);
        ranges.add(max - min);
    }
```

```java
    ArrayList<double[]> normalizedDatas = new ArrayList<>();
    double[] normD;
    for (double[] ds : datas) {
        normD = new double[ds.length];
        for (int i = 0; i < ds.length; i++) {
            normD[i] = (ds[i] - minVals.get(i)) / ranges.get(i);
        }
        normalizedDatas.add(normD);
    }

    normalizedTrainData.put("minVals", minVals);
    normalizedTrainData.put("ranges", ranges);
    normalizedTrainData.put("datas", normalizedDatas);
    normalizedTrainData.put("labels", trainData.get("labels"));

    return normalizedTrainData;
}
```

3.1.3 테스트 데이터 정규화 시키는 메소드

```java
// 테스트 할 데이터 정규화
// p : double[] testData, HashMap<String, ArrayList> normalizedTrainData
@SuppressWarnings({ "unchecked", "rawtypes" })
public static double[] normalizeTestData(double[] testData, HashMap<String, Array
    double[] normalizedTestData = new double[testData.length];
    ArrayList<Double> minVals = normalizedTrainData.get("minVals");
    ArrayList<Double> ranges = normalizedTrainData.get("ranges");
    for (int i = 0; i < testData.length; i++) {
        normalizedTestData[i] = (testData[i] - minVals.get(i)) / ranges.get(i);
    }
    return normalizedTestData;
}
```

3.2 kNNMain.java

3.2.1 main()

```
@SuppressWarnings("rawtypes")
public static void main(String[] args) {
    HashMap<String, ArrayList> trainData = kNN.trainFromFile("D:\\Kwon\\자원\\for A
    trainData = kNN.normalizeTrainData(trainData);

    Scanner s = new Scanner(System.in);
    double[] testData = new double[3];

    System.out.print("일년동안 비행기로 이동한 거리(mile) : ");
    testData[0] = s.nextDouble();

    System.out.print("하루중 게임으로 보내는 시간 비율 : ");
    testData[1] = s.nextDouble();

    System.out.print("일주일동안 먹는 아이스크림 양(L) : ");
    testData[2] = s.nextDouble();

    System.out.print("k : ");
    int k = s.nextInt();

    testData = kNN.normalizeTestData(testData, trainData);

    String result = kNN.do_kNN(testData, trainData, k);
    System.out.println(result);

    s.close();
}
```

3.3 실행

```
일년동안 비행기로 이동한 거리(mile) : 90000
하루중 게임으로 보내는 시간 비율 : 10
일주일동안 먹는 아이스크림 양(L) : 4
k : 7
{보통=0, 인기많음=1, 별로=6}
별로
```